

Introduction to Digital Forensics

Linux command-line foundations · evidence hashing · file signatures

Stefanus Eko Prasetyo

Lab-1 · hands-on session

```
analyst@kali: ~  
$ whoami  
forensic-analyst  
$ echo "every trace tells a story"  
every trace tells a story
```

Today's roadmap

1

What digital forensics is

definition, process, and where it is used

2

The digital trail

what every action on a system leaves behind

3

The Linux toolbox

12 commands that carry a whole investigation

4

Hashing for integrity

md5sum and sha1sum — proof that evidence is untouched

5

Magic bytes & file headers

why extensions lie and content does not

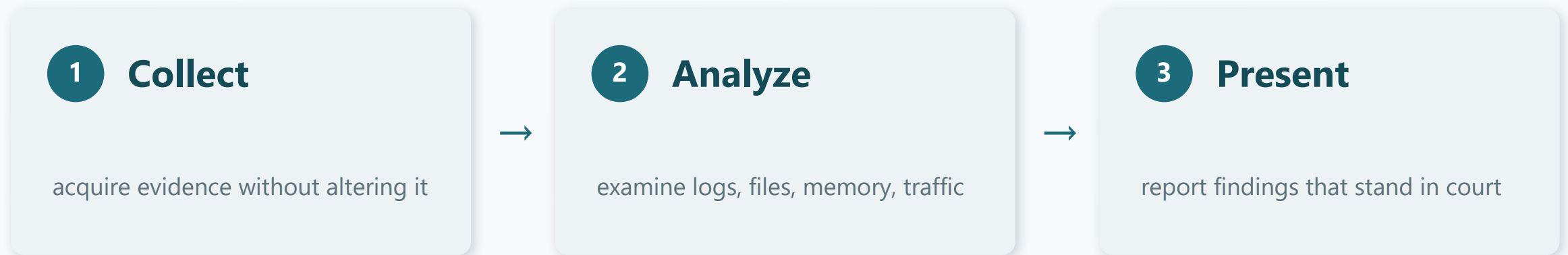
6

Challenges & working paper

three hands-on challenges + how to document them

What is digital forensics?

Using technology to **gather evidence**, **investigate it**, and **present the findings** in a legal case.



The common goal: collect evidence that can be used to prosecute suspects in a court of law.

Where it is used

Investigating cyber attacks

Determine the scope and source of a breach, then harden defenses against the next one.

Threat detection & response

Proactively hunt and mitigate security threats before they become incidents.

Data recovery

Recover data that was stolen or deleted during an attack.

Criminal investigations

Identify suspects, establish motive, and link suspects to specific crimes.

The digital trail

Every digital action leaves evidence.

The investigator's job is to find that trail — and to interpret it correctly.

- **File access timestamps**
when files were created, changed, opened
- **Network connection logs**
who talked to whom, on which port
- **Process execution records**
what ran, under which account
- **Browser history**
searches, downloads, form data
- **Deleted file remnants**
deleted is not gone — traces persist

The Linux toolbox

Most forensic tooling lives on Linux. Twelve commands carry an entire first investigation:

Group	Commands	Why it matters
Navigate & inspect	ls · cd · file	move around fast; identify what a file really is
Read content	cat · strings	read text; pull readable text out of binaries
Search	grep · find	hunt patterns in logs; locate files by name, type, or time
Prove integrity	md5sum · sha1sum	show evidence was not tampered with
Go low-level	xxd · hexedit	hex dumps; patch bytes such as broken headers
Live system	ps · netstat	snapshot running processes and connections

Navigate & inspect — ls, cd, file

```
analyst@kali: ~  
$ ls  
Desktop  Documents  Downloads  Music  Pictures  
  
$ ls -la /var/log  
drwxr-xr-x 14 root root  auth.log  kern.log  syslog  
  
$ file suspect.png  
suspect.png: PNG image data, 562 x 424, 8-bit/color  
  
$ file image.jpg  
image.jpg: ELF 64-bit LSB pie executable, x86-64  
# a jpg that is actually a Linux executable!
```

Reading the signals

- ▶ file identifies content by signature — it never trusts the extension
- ▶ an image.jpg that reports ELF is malware wearing a costume
- ▶ ls -la also reveals hidden dotfiles and permissions
- ▶ cd moves the rest of the toolkit into position

Read content — cat, strings

```
analyst@kali: ~  
  
$ cat note.txt  
Meet at the usual place. Bring the drive.  
  
$ strings firmware.bin | head -5  
/lib64/ld-linux-x86-64.so.2  
http://updates.example.net/config  
GVsbG8sIHdvcmxkIQ==  
  
$ strings -n 8 malware.bin | grep http  
http://updates.example.net/config
```

When files are not text

- ▶ cat prints plain text files in full
- ▶ strings extracts human-readable sequences from any binary
- ▶ malware configs, URLs, and base64 blobs often surface this way
- ▶ -n 8 keeps only strings of 8+ characters — less noise

Search — grep, find

```
analyst@kali: ~  
$ grep "Failed password" /var/log/auth.log  
Jan 12 08:14:02 sshd[2011]: Failed password for admin  
from 203.0.113.77 port 51234 ssh2  
  
$ grep -c "203.0.113.77" server.log  
23  
  
$ find /home -mmin -30  
/home/analyst/notes.txt  
  
$ find / -name "*.jpg" 2>/dev/null  
/opt/staging/logo.jpg /tmp/drop/img002.jpg
```

Finding the needle

- ▶ grep filters huge logs down to the lines that matter
- ▶ count hits with -c, ignore case with -i
- ▶ find locates files by name, type, size, or modification time
- ▶ -mmin -30 = changed in the last 30 minutes — fresh evidence

Live system — ps, netstat

```
analyst@kali: ~  
$ ps aux | head -4  
USER    PID  %CPU  COMMAND  
root      1   0.0  systemd  
analyst 2841 12.9  firefox  
analyst 3102  0.1 miner  <- unfamiliar name?  
  
$ netstat -tn  
Proto Local                Foreign              State  
tcp    192.168.0.106:44300  93.184.220.29:http  ESTABLISHED  
tcp    192.168.0.106:58308  52.39.122.167:https ESTABLISHED
```

Snapshot the living system

- ▶ ps lists running processes — spot unfamiliar names and odd parents
- ▶ netstat shows active connections and their states
- ▶ unusual foreign addresses or ports deserve a second look
- ▶ this is live-response territory: memory comes later (Lab 07)

Hashing — proof that evidence is untouched

A hash turns any input into a fixed-length fingerprint. Same content in, same hash out — change one bit and the fingerprint changes completely.

```
analyst@kali: ~  
  
$ md5sum evidence.dd  
8ddd8be4b179a529afa5f2ffae4b9858  evidence.dd  
  
$ sha1sum evidence.dd  
a0b65939670bc2c010f4d5d6a0b3e4e4590fb92b  evidence.dd  
  
$ echo x >> copy.dd    # one byte added to a COPY  
$ md5sum copy.dd  
b1946ac92492d2347c6235b4d2611184  copy.dd  
# completely different hash - the avalanche effect
```

Hash at acquisition

record MD5/SHA1 the moment evidence is received

Verify after analysis

re-hash and compare before you report anything

Always work on copies

the original stays sealed — you analyze the copy

Magic bytes & file headers

A file signature (magic bytes) is a fixed sequence of bytes at the start of a file that identifies its real type — regardless of its extension.

Format	First bytes (hex)	Notes
JPEG	FF D8 FF E0	photos; EXIF metadata lives here
PNG	89 50 4E 47 0D 0A 1A 0A	8-byte signature — today's challenge target
PDF	25 50 44 46 2D	reads as %PDF-
BMP	42 4D	reads as BM
ZIP	50 4B 03 04	also the prefix of .docx and .xlsx

```
analyst@kali: ~  
$ xxd logo.png | head -1  
00000000: 8950 4e47 0d0a 1a0a  
          0000 000d 4948 4452  
  
# 8950 4e47 = the PNG signature  
# 4948 4452 = IHDR, image header
```

**Extensions can be renamed in one click.
Bytes cannot lie.**

Repairing a broken file header



```
$ file challenge_copy.png
challenge_copy.png: data
$ xxd challenge_copy.png | head -1
00000000: 0000 0000 0000 000d 4948 4452
$ hexedit challenge_copy.png # restore 8950 4e47 0d0a 1a0a, save
$ file challenge_copy.png
challenge_copy.png: PNG image data, 1400 x 788, 8-bit/color RGB
```

analyst@kali: ~

Your challenges

Challenge A

challenge.png

files/challenge.png

- ▶ the PNG header is damaged
- ▶ spot the wrong bytes
- ▶ repair a copy, recover the flag

from the original lab

Challenge B

challenge_extra.png

files/challenge_extra.png

- ▶ the file lies about its type
- ▶ deep inside it still whispers PNG
- ▶ restore the true signature

a new twist on the same idea

Challenge C — new

access_big.log

files/ (log + update.bin)

- ▶ 8,600 lines, one attacker
- ▶ a secret is hidden URL-encoded
- ▶ a binary holds one more

grep and strings power hunt

Flag format: `ctf{xxxx}` hints and step-by-step guidance live in your working paper

Working paper & evidence rules

1 Pre-lab before the session

answer the understanding check in the working paper first

2 Check off every step

tick each step as you complete it — in order

3 Number your evidence

screenshots and outputs get B-01, B-02, B-03 and land in the evidence log

4 Hash what you touch

fill the integrity form before and after analysis

5 Reflect

close each lab with findings, lessons, and obstacles

```
analyst@kali: ~  
  
# evidence log entry  
[B-04] 2026-09-30 10:12  
xxd challenge_extra.png | head -1  
00000000: ffd8 ffe0 0000 000d  
-> header claims JPEG (FF D8 FF E0)  
-> IHDR at offset 12 says otherwise
```

No evidence, no answer.

every claim in your report must trace back to a numbered exhibit

KEY TAKEAWAYS

01

The CLI is your microscope

ls, cat, strings, grep, find, file, xxd — twelve tools, one investigation

02

Hashes prove integrity

hash at acquisition, verify after analysis, always work on copies

03

Extensions lie, bytes don't

file and xxd reveal what a file really is, whatever its name says

04

Document everything

numbered evidence, commands, and timestamps make findings repeatable

Clone this lab and get to work.



analyst@kali: ~

```
$ git clone https://github.com/stprasst/digital-forensic-lab
$ cd digital-forensic-lab/Lab\ 01/files
$ ls
challenge.png  images/  server.log  update.bin
```